

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ-UTFPR
RELATÓRIO DE PROJETO

**INTRODUÇÃO A PRÁTICAS DE LABORATÓRIO EM ELETRICIDADE E
ELETRÔNICA**

GENIUS - JOGO DA MEMÓRIA COM ARDUINO

Julia Ortolani de Souza-2877562

Nathalia Fujiwara Paulin-287600

Rafaela Vedolin Feistler-2877627

Professor: João Alberto Fabro

Curitiba

2026

1. RESUMO

Esse relatório tem como objetivo documentar e explicar o projeto desenvolvido para a matéria Introdução a Práticas de Laboratório em Eletricidade e Eletrônica, ministrada pelo professor João Alberto Fabro na UTFPR Campus Curitiba. A proposta desenvolvida é replicar o funcionamento do brinquedo de jogo da memória Genius, da empresa de Estrela, utilizando materiais e conceitos apresentados durante as aulas. Assim, ao final do projeto, foi criado um brinquedo funcional que testa a memória do usuário, através de sequências de luzes coloridas, que vão aumentando de dificuldade com o passar do jogo.

2. INTRODUÇÃO

O Genius, da Estrela, foi um brinquedo de jogo da memória muito popular nos anos 80, sendo considerado o primeiro jogo eletrônico lançado no Brasil e ficando conhecido como “o computador que brinca”. Ele possui quatro botões que acendem com uma luz colorida, uma vermelha, uma verde, uma amarela e uma azul (Figura 1). O objetivo do jogo é replicar a sequência mostrada pelas luzes na ordem correta, a qual vai aumentar a cada acerto.



Figura 1- Genius, da Estrela

Fonte: Site oficial da Estrela

3. MATERIAIS E MÉTODOS

Para o desenvolvimento do projeto, o funcionamento do jogo foi adaptado, para que ele possua cinco níveis de dificuldade consecutivos. Assim, em uma mesma sequência de cores, quanto mais acertos, mais rápida e mais “comprida” ficará a sequência mostrada. Cada ação é acompanhada de sons produzidos por um pequeno buzzer e um Arduino Uno é utilizado para controlar o funcionamento geral do brinquedo.

Os materiais utilizados foram:

- 1 Arduino Uno+cabo de alimentação USB;
- 4 LEDs grandes (1 amarelo, 1 vermelho, 1 verde e 1 azul);
- 4 push-buttons coloridos (1 amarelo, 1 vermelho, 1 verde e 1 azul);
- 5 LEDs vermelhos de tamanho comum;
- 1 buzzer passivo AC-1205G;
- 4 resistores de 10k Ω (para pull-down);
- 5 resistores de 220 Ω (para os LEDs vermelhos);
- 4 resistores de 330 Ω (para os LEDs grandes);
- 1 placa de circuito impresso perfurada 10x15cm;
- 1 ferro de solda+estanho;
- Alguns jumpers.

O código utilizado no Arduino é (imagens de autoria própria feitas na plataforma ArduinoIDE):

```
/*
* PROJETO FINAL - LABORATÓRIO DE ELETRÔNICA
* JOGO DA MEMÓRIA "GENIUS" COM 5 FASES DE DIFICULDADE
* VERSÃO: Debounce por Software (Sem Capacitores)
* Curso: Engenharia - 1º Período
*/

// Botões (Entradas com resistores de Pull-Down externos)
const int BTN_VERDE = 2;
const int BTN_VERMELHO = 3;
const int BTN_AZUL = 4;
const int BTN_AMARELO = 5;

// LEDs Principais do Jogo (Saídas)
const int LED_VERDE = 6;
const int LED_VERMELHO = 7;
const int LED_AZUL = 8;
const int LED_AMARELO = 9;

//Buzzer
const int BUZZER = 10;

// LEDs Indicadores de Fase (Usando pinos analógicos como digitais)
const int LED_FASE1 = A0;
const int LED_FASE2 = A1;
const int LED_FASE3 = A2;
const int LED_FASE4 = A3;
const int LED_FASE5 = A4;
```

```

// --- VARIÁVEIS GLOBAIS DO JOGO ---
const int MAX_SEQUENCIA = 12; // Máximo de rodadas para zerar a fase 5
int sequencia[MAX_SEQUENCIA]; // Vetor que guarda a sequência de cores gerada
int rodada_atual = 0; // Quantos passos a sequência atual possui
int fase_atual = 1; // Fase do jogo (1 a 5)
int velocidade = 1000; // Tempo em milissegundos que o LED fica aceso

const int ledsJogo[4] = {
    LED_VERDE,
    LED_VERMELHO,
    LED_AZUL,
    LED_AMARELO
};

// --- CONFIGURAÇÕES INICIAIS ---
void setup() {
    // Configurando os pinos dos Botões como Entradas
    pinMode(BTN_VERDE, INPUT);
    pinMode(BTN_VERMELHO, INPUT);
    pinMode(BTN_AZUL, INPUT);
    pinMode(BTN_AMARELO, INPUT);

    // Configurando os pinos dos LEDs do jogo como Saídas
    pinMode(LED_VERDE, OUTPUT);
    pinMode(LED_VERMELHO, OUTPUT);
    pinMode(LED_AZUL, OUTPUT);
    pinMode(LED_AMARELO, OUTPUT);

    // Configurando os pinos dos LEDs de Fase como Saídas
    pinMode(LED_FASE1, OUTPUT);
    pinMode(LED_FASE2, OUTPUT);
    pinMode(LED_FASE3, OUTPUT);
    pinMode(LED_FASE4, OUTPUT);
    pinMode(LED_FASE5, OUTPUT);

    // Configurando Buzzer
    pinMode(BUZZER, OUTPUT);

    // Inicializa o gerador de números aleatórios com base em uma leitura flutuante do pino A5
    randomSeed(analogRead(A5));

    // Executa uma animação de início para testar se tudo está funcionando
    animacaoInicio();
}

// --- LOOP PRINCIPAL DO JOGO ---
void loop() {
    // 1. Atualiza visualmente quais LEDs de fase devem estar acesos
    atualizarLedsFase();

    // 2. Verifica se a fase mudou e ajusta a velocidade/dificuldade
    atualizarDificuldade();

    // 3. Adiciona uma nova cor aleatória à sequência
    sequencia[rodada_atual] = ledsJogo[random(0, 4)];
    rodada_atual++;
}

```

```

// 4. Mostra a sequência atual piscando os LEDs e tocando o som correspondente
for (int i = 0; i < rodada_atual; i++) {
    int pinoLed = sequencia[i];
    int frequenciaSom = obterFrequencia(pinoLed);

    digitalWrite(pinoLed, HIGH);
    tone(BUZZER, frequenciaSom, velocidade - 100);
    delay(velocidade);
    digitalWrite(pinoLed, LOW);
    delay(200); // Pequena pausa entre os LEDs para não emendar as cores
}

// 5. Aguarda a resposta do jogador para cada passo da sequência
for (int i = 0; i < rodada_atual; i++) {
    int botaoPressionado = aguardarBotao();

    // Traduz o botão pressionado para o respectivo pino do LED
    int ledCorrespondente;

    switch(botaoPressionado) {
        case BTN_VERDE:
            ledCorrespondente = LED_VERDE;
            break;

        case BTN_VERMELHO:
            ledCorrespondente = LED_VERMELHO;
            break;

        case BTN_AZUL:
            ledCorrespondente = LED_AZUL;
            break;
    }
}

```

```

    case BTN_AMARELO:
        ledCorrespondente = LED_AMARELO;
        break;
}

// Efeito visual e sonoro do botão que o jogador apertou
digitalWrite(ledCorrespondente, HIGH);
tone(BUZZER, obterFrequencia(ledCorrespondente), 200);
delay(300);
digitalWrite(ledCorrespondente, LOW);

// Se o jogador errou o botão da sequência...
if (ledCorrespondente != sequencia[i]) {
    gameOver();
    return; // Sai do loop principal e reinicia o jogo do zero
}
}

// Se o jogador acertou tudo da rodada e atingiu o limite da Fase 5: Vitória Máxima!
if (rodada_atual >= MAX_SEQUENCIA) {
    vitoriaTotal();
    return;
}

delay(1000); // Pausa de 1 segundo antes de começar a próxima rodada
}

// --- FUNÇÕES AUXILIARES ---

// Aguarda o jogador pressionar algum botão com DEBOUNCE POR SOFTWARE integrado
int aguardarBotao() {
    while (true) {
        // Verifica Botão Verde
        if (digitalRead(BTN_VERDE) == HIGH) {
            delay(50); // Aguarda a trepidação inicial passar
            while(digitalRead(BTN_VERDE) == HIGH); // Trava o código enquanto o jogador segurar o botão
            delay(50); // Aguarda a trepidação de soltar o botão
            return BTN_VERDE;
        }

        // Verifica Botão Vermelho
        if (digitalRead(BTN_VERMELHO) == HIGH) {
            delay(50);
            while(digitalRead(BTN_VERMELHO) == HIGH);
            delay(50);
            return BTN_VERMELHO;
        }

        // Verifica Botão Azul
        if (digitalRead(BTN_AZUL) == HIGH) {
            delay(50);
            while(digitalRead(BTN_AZUL) == HIGH);
            delay(50);
            return BTN_AZUL;
        }
    }
}

```

```

// Verifica Botão Amarelo
if (digitalRead(BTN_AMARELO) == HIGH) {
    delay(50);
    while(digitalRead(BTN_AMARELO) == HIGH);
    delay(50);
    return BTN_AMARELO;
}

delay(10); // Pequena folga para o processador do Arduino
}

// Retorna uma frequência de som em Hz diferente para cada LED do jogo
int obterFrequencia(int pinoLed) {
    switch (pinoLed) {
        case LED_VERDE: return 262; // Dó
        case LED_VERMELHO: return 294; // Ré
        case LED_AZUL: return 330; // Mi
        case LED_AMARELO: return 349; // Fá
        default: return 440;
    }
}

// Atualiza a barra física de LEDs indicadores de fase
void atualizarLedsFase() {
    digitalWrite(LED_FASE1, (fase_atual >= 1) ? HIGH : LOW);
    digitalWrite(LED_FASE2, (fase_atual >= 2) ? HIGH : LOW);
    digitalWrite(LED_FASE3, (fase_atual >= 3) ? HIGH : LOW);
    digitalWrite(LED_FASE4, (fase_atual >= 4) ? HIGH : LOW);
    digitalWrite(LED_FASE5, (fase_atual >= 5) ? HIGH : LOW);
}

```



```
// Controla a passagem de fases baseado na quantidade de acertos (rodadas)
void atualizarDificuldade() {
    if (rodada_atual < 3) {
        fase_atual = 1;
        velocidade = 1000;
    }
    else if (rodada_atual < 5) {
        if (fase_atual == 1) somSubiuDeFase();
        fase_atual = 2;
        velocidade = 800;
    }
    else if (rodada_atual < 7) {
        if (fase_atual == 2) somSubiuDeFase();
        fase_atual = 3;
        velocidade = 600;
    }
    else if (rodada_atual < 9) {
        if (fase_atual == 3) somSubiuDeFase();
        fase_atual = 4;
        velocidade = 400;
    }
    else {
        if (fase_atual == 4) somSubiuDeFase();
        fase_atual = 5;
        velocidade = 250;
    }
}
```

```

// Som rápido emitido quando o jogador muda de fase
void somSubiuDeFase() {
    for (int i = 0; i < 3; i++) {
        tone(BUZZER, 523, 100);
        delay(150);
    }
}

// Animação que pisca os LEDs sequencialmente ao ligar o circuito
void animacaoInicio() {
    int leds[] = {LED_VERDE, LED_VERMELHO, LED_AZUL, LED_AMARELO};
    for (int i = 0; i < 4; i++) {
        digitalWrite(leds[i], HIGH);
        tone(BUZZER, 300 + (i * 100), 100);
        delay(150);
        digitalWrite(leds[i], LOW);
    }
    delay(500);
}

// Ações executadas quando o jogador erra a sequência
void gameOver() {
    fase_atual = 0;
    atualizarLedsFase();

    for(int i = 0; i < 3; i++) {
        digitalWrite(LED_VERDE, HIGH);
        digitalWrite(LED_VERMELHO, HIGH);
        digitalWrite(LED_AZUL, HIGH);
        digitalWrite(LED_AMARELO, HIGH);

        tone(BUZZER, 150, 200);

        delay(250);
    }
}

```

```

    digitalWrite(LED_VERDE, LOW);
    digitalWrite(LED_VERMELHO, LOW);
    digitalWrite(LED_AZUL, LOW);
    digitalWrite(LED_AMARELO, LOW);

    delay(250);
}

rodada_atual = 0;
fase_atual = 1;

delay(2000);
}

// Comemoração se o jogador passar de todas as fases (Acertar 12 seguidas)
void vitoriaTotal() {

    for (int i = 0; i < 5; i++) {
        digitalWrite(LED_VERDE, HIGH);
        digitalWrite(LED_VERMELHO, HIGH);
        digitalWrite(LED_AZUL, HIGH);
        digitalWrite(LED_AMARELO, HIGH);
        tone(BUZZER, 523, 200);
        delay(250);

        digitalWrite(LED_VERDE, LOW);
        digitalWrite(LED_VERMELHO, LOW);
        digitalWrite(LED_AZUL, LOW);
        digitalWrite(LED_AMARELO, LOW);
        tone(BUZZER, 659, 200);
        delay(250);
    }

    rodada_atual = 0;
    fase_atual = 1;
    delay(2000);
}

```

A montagem segue o diagrama esquemático elétrico mostrado na Figura 2.

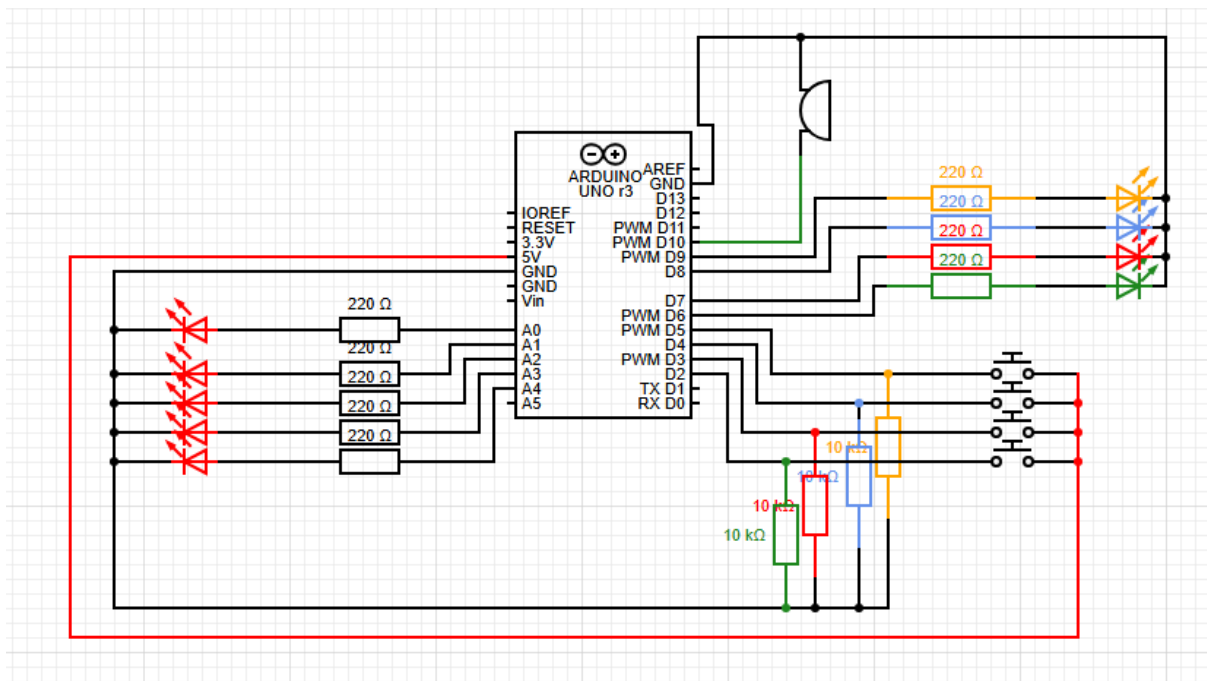


Figura 2- Esquemático elétrico do projeto

Fonte: Autoria própria, por meio do programa Circuit Diagram

4. RESULTADOS E DISCUSSÕES

Primeiramente, o circuito e programa foram testados no programa de simulação Tinkercad, como mostra a Figura 3. Após isso, o circuito foi montado em uma Protoboard, para testar o funcionamento físico, como mostra a Figura 4. Finalmente, quando tudo estava funcionando corretamente, os componentes foram soldados na placa de circuito impressa perfurada (Figura 5).

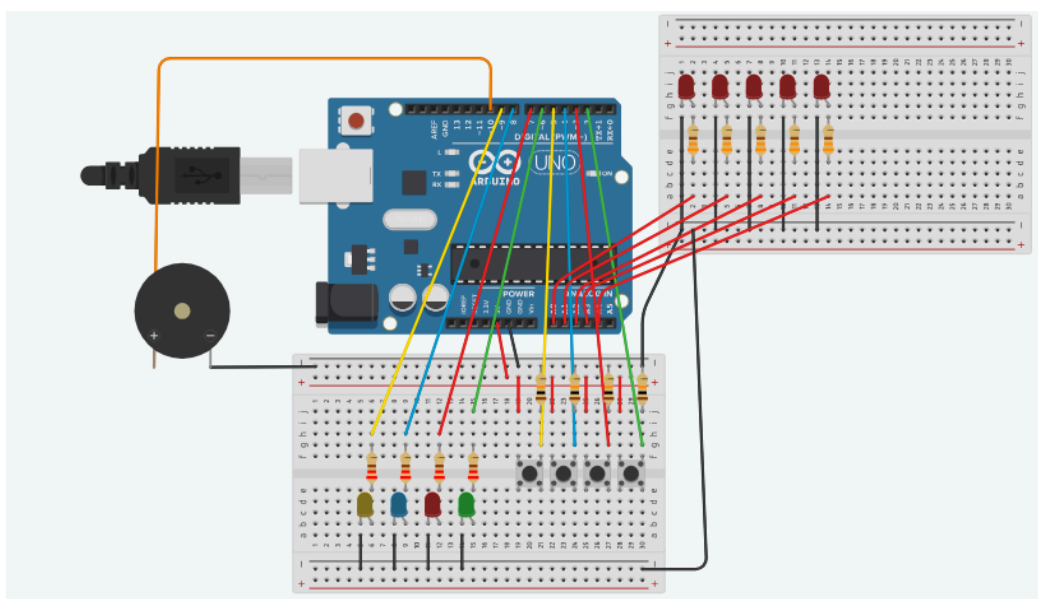


Figura 3- Simulação do projeto

Fonte: Autoria própria, por meio do programa Tinkercad

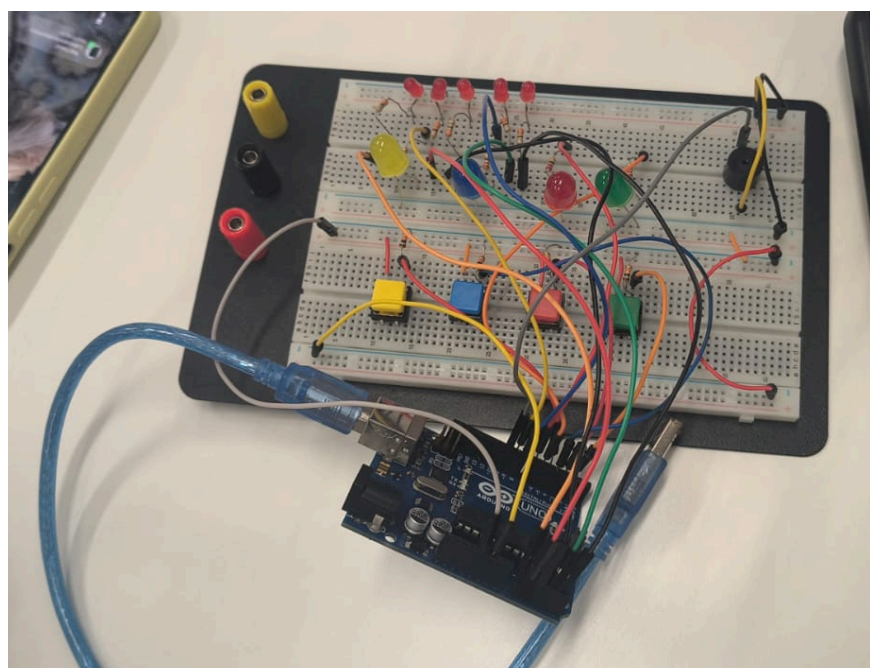


Figura 4- Projeto na Protoboard

Fonte: Autoria própria

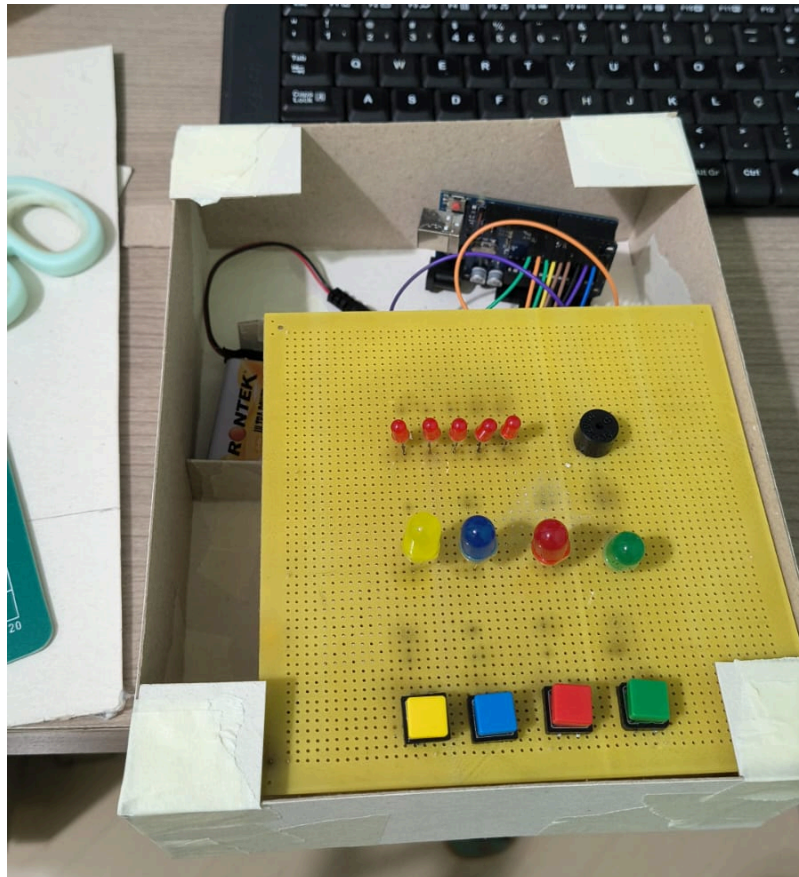


Figura 5- Projeto soldado da placa perfurada

Fonte: Autoria própria

5. CONCLUSÃO

Por meio deste trabalho foi possível aplicar diversos conhecimentos adquiridos em aula de forma a produzir um brinquedo funcional, tendo como base o Genius. A parte principal para o desenvolvimento desse projeto foi a programação utilizada no Arduino, que permitiu a interatividade do projeto (pelos botões, buzzer e luzes). De forma geral, o resultado final foi satisfatório e divertido, com um funcionamento idêntico ao visualizado nas simulações.

REFERÊNCIAS

- <https://exame.com/marketing/estrela-relanca-genius-e-boca-rica-em-setembro/>
- <https://www.estrela.com.br/jogo-genius-estrela/p?srsItid=AfmBOooIMX8MzXUXEQAnYHRza9xPj2UI5KdrxKMJu10K3C-U2dDCJeM>
- <https://www.estrela.com.br/institucional/nossa-historia>
- <https://www.circuit-diagram.org/editor/>